Project 2: Real-Time IoT Device Monitoring & Cloud Dashboarding Platform

Proposed by: https://www.sdataway.com/

Key contacts: Nabil Abdennadher, Raoul Dupuis, Nizar Bouchedakh (SDATAWAY)

1. Context and Motivation

Modern connected devices (embedded systems, edge gateways, industrial sensors) require continuous monitoring of their operational health: uptime, temperature, power consumption, memory usage, and error states. Organizations increasingly need lightweight, cloud-native pipelines to collect this telemetry in real time and turn it into actionable insight through dashboards and analytics.

This project asks students to design and implement an end-to-end IoT monitoring Cloud based solution, from a simulated embedded device generating telemetry, through an edge data collector, into a cloud time-series backend, and finally into a visualization and analysis layer. Beyond the technical build, the project is structured to develop project management skills: gathering and formalizing requirements, translating them into a feature backlog, assessing feasibility, and tracking implementation against a plan.

2. Objectives

1. Design and implement a data pipeline that pushes real-time device monitoring data to the cloud.
2. Design and implement a dashboarding/analytics solution to visualize and monitor this data.
3. Practice requirements engineering and project management: capturing client needs, converting them into features, estimating feasibility/effort, and tracking delivery.
4. (Stretch goal) Run an improvement loop: identify issues/limitations in the first implementation and propose/implement a remediation plan.

3. System Overview

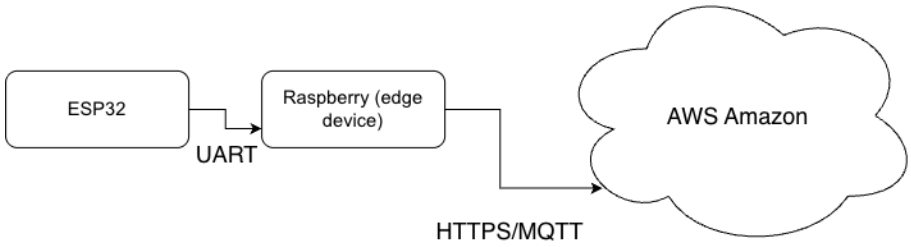


Fig 1.2: End-to-end IoT monitoring Cloud based solution

3.1 Simulated Device (Data Generator)

As shown in Fig.1.2, a Raspberry Pi or ESP32 dev board runs firmware/script that generates simulated monitoring data and streams it over a serial (UART/USB) connection. Suggested data fields:

Field	Type	Notes
Timestamp` (device-local, optional)	int/string	Device may or may not have accurate time
device_id	string	Identifies the simulated unit
uptime_seconds	int	Monotonically increasing, resets on simulated reboot
temperature_c	float	Simulated with noise/drift, occasional spikes
power_consumption_mw	float	Simulated load variation
heap_free_bytes	int	Simulated memory usage/fragmentation
device_status	Enum	e.g. `OK`, `DEGRADED`, `ERROR`, `REBOOTING`
error_code	int/string	Populated when status indicates an error

Data should be emitted at a configurable interval (e.g. every 1–5 seconds) in a well-defined serial format (e.g. JSON lines, or CSV with a defined schema).

3.2 Edge Collector (Raspberry Pi, in Fig. 1.2)

1. Acts as the gateway between the device and the cloud:
2. Reads and parses the serial stream.
3. Validates/cleans malformed data.
4. Adds a reliable timestamp (edge-side, NTP-synced) and any needed metadata (edge/gateway ID).
5. Buffers data locally if connectivity is lost (store-and-forward).
6. Pushes formatted data to AWS over the internet (protocol choice — HTTPS/REST, MQTT, or AWS IoT Core — is part of the students' technology evaluation).

3.3 Cloud Ingestion & Storage (AWS Amazon in Fig. 1.2)

1. An ingestion endpoint (e.g. AWS IoT Core, API Gateway + Lambda, or Kinesis) receives data from the edge device.
2. Data is persisted in a time-series-oriented store. Candidates to evaluate: Amazon Timestream, InfluxDB (self-hosted or InfluxDB Cloud), TimescaleDB on RDS, or DynamoDB with time-based keys.
3. Students should document the tradeoffs (cost, managed vs self-hosted, query capabilities, retention/downsampling) that justified their choice.

3.4 Dashboarding & Analysis

A visualization layer to monitor device health and explore historical trends. Candidates to evaluate: Grafana (self-hosted or Amazon Managed Grafana), Amazon QuickSight, or a custom web dashboard (e.g. React + charting library reading from an API).

Expected capabilities:

1. Live/near-real-time view of current device status per device.
2. Historical trends (temperature, power, heap usage over time).
3. Alerting/flagging on error states or abnormal readings (stretch goal).
4. Multi-device view if multiple simulated devices are run in parallel.

4. Project Phases

4.1 Phase 0 — Requirements & Design

1. Act as "client" stakeholder(s); students conduct requirement-gathering interviews/sessions.

2. Students produce a requirements document and translate it into a prioritized feature backlog.
3. Students propose a technology stack with a brief feasibility/trade-off analysis per component (device-to-edge protocol, edge-to-cloud protocol, storage, dashboard).

4.2 Phase 1 — Real-Time Data Pipeline (Device → Edge → Cloud)

1. Implement the simulated data generator (ESP32/Raspberry Pi).
2. Implement the edge collector (serial ingestion, timestamping, formatting, store-and-forward buffering).
3. Implement cloud ingestion and time-series storage.

4.3 Phase 2 — Dashboard & Data Analysis

1. Implement the dashboard against the time-series store.
2. Add basic analysis: trends, thresholds, simple anomaly flags.

4.4 Phase 3 (Stretch) — Enhancement Loop

1. Students review their own system: identify limitations (e.g. data loss scenarios, security gaps, scaling limits, dashboard usability).
2. Produce a short improvement plan with prioritized fixes.
3. Implement a subset of the improvements, time permitting.

5. Skills Developed

5.1 Technical

1. Embedded/edge programming (serial communication, data formatting)
2. Cloud architecture (IoT ingestion, time-series storage, security/auth for device-to-cloud communication)
3. Data visualization and basic analytics
4. System testing (functional, resilience/reconnection, load)

5.2 Project management

1. Requirements elicitation and translation into features
2. Feasibility assessment and technology trade-off analysis
3. Backlog management and progress tracking against a plan

4. Root-cause analysis and structured improvement planning

6. Deliverables

1. Requirements & architecture documentation/presentation
2. Working device data generator (ESP32 or Raspberry Pi)
3. Cloud ingestion + time-series storage, deployed and documented
4. Dashboard with live + historical views
5. Test report (functional + resilience tests)

7. Evaluation Criteria

1. Requirements analysis & documentation quality: 15%
2. Architecture & technology choice justification : 20%
3. Pipeline implementation (device → edge → cloud): 35%
4. Dashboard & analysis implementation: 20%
5. Testing rigor: 10%

8. Open Questions for Review

1. Should security (device auth, encryption in transit) be a graded requirement or optional enhancement?